



JSP + MVC

1

什么是JSP

2

MVC设计模式

3

一个综合案例

1

什么是JSP

2

MVC设计模式

3

一个综合案例

什么是JSP

□ JSP概述

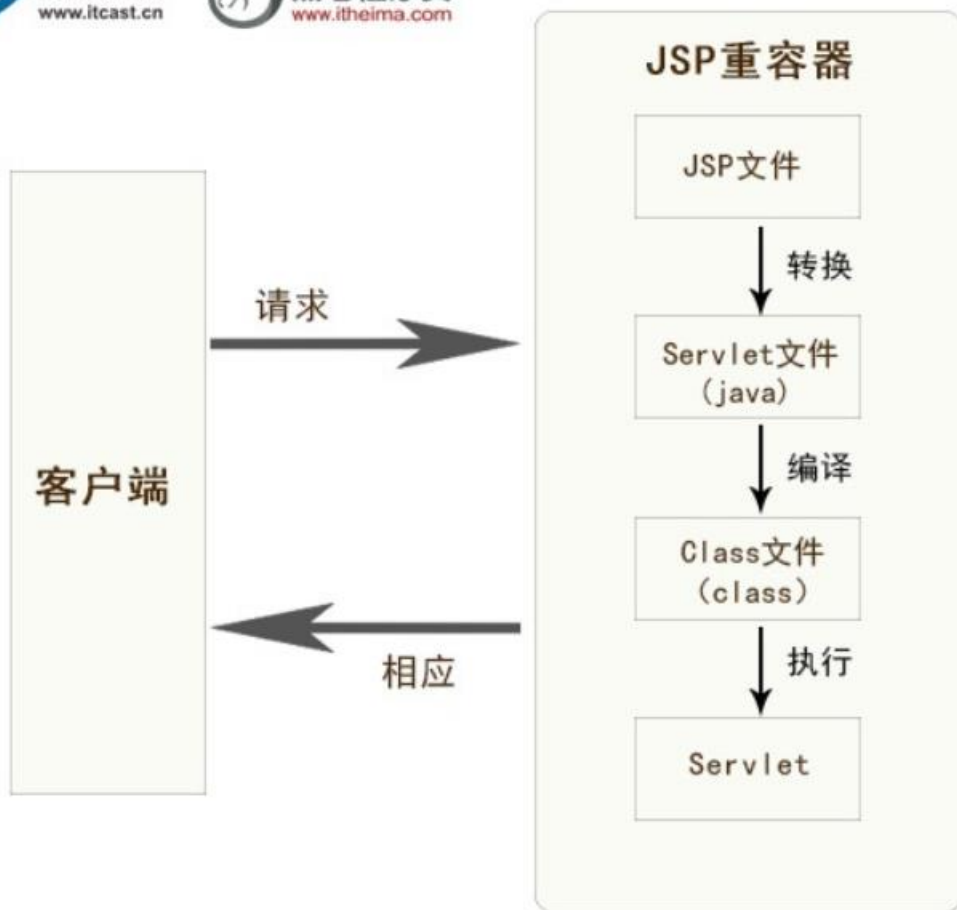
- Java Server Pages (Java服务器端页面)
- 一个简化版的Servlet, 克服了原先Servlet的一些缺点
 1. Servlet需要进行配置, 不方便维护
 2. Servlet往页面回写的内容不方便与网页页面的html标签结合!!!

□ JSP特点

- 可以在html中嵌入Java代码 (定义变量, 并将其嵌入到html标签中);
- JSP其本质是一个Servlet, 封装了诸如: request、response等内置对象;
- 在实际开发中, JSP重在显示页面!!!

什么是JSP

□ JSP的运行原理



回忆Servlet:

Servlet是运行在服务端的java代码，
运行前要编译生成.class文件

注：只有服务端

再看JSP:

1. JSP也是一个Servlet（Java类），
在运行前也要编译生成.class文件
（浏览器访问JSP时生成）

2. JSP页面中的html 标签利用 java
代码进行输出

注：前端 + 服务端

什么是JSP

□ JSP的运行原理

 jsp01_jsp.class	2022/6/9 20:55	CLASS 文件	6 KB
 jsp01_jsp.java	2022/6/9 20:55	JAVA 文件	5 KB

```
out.write("\r\n");
out.write("<!DOCTYPE html>\r\n");
out.write("<html>\r\n");
out.write("<head>\r\n");
out.write("<meta charset=\"UTF-8\">\r\n");
out.write("<title>Insert title here</title>\r\n");
out.write("</head>\r\n");
out.write("<body>\r\n");
out.write("    <h1>Hello, JSP!</h1>\r\n");
out.write("</body>\r\n");
out.write("</html>");
```

前端页面！！！！

```
public void _jspInit() {
}

public void _jspDestroy() {
}

public void _jspService(final javax.servlet.http.HttpServletRequest request, final javax.servlet.http.HttpServletResponse response)
    throws java.io.IOException, javax.servlet.ServletException {

    final java.lang.String _jspx_method = request.getMethod();
    if (!"GET".equals(_jspx_method) && !"POST".equals(_jspx_method) && !"HEAD".equals(_jspx_method) && !javax.servlet.DispatcherType.ERROR.equals(request.getDispatcherType())) {
        response.sendError(HttpServletResponse.SC_METHOD_NOT_ALLOWED, "JSP 只允许 GET、POST 或 HEAD。Jasper 还允许 OPTIONS");
        return;
    }
}
```

后端Servlet创建、处理请求、销毁的java代码！！！！

什么是JSP

□ 如何在JSP中嵌入Java代码（JSP脚本元素），让html标签与变量相结合

1. 表达式标签

<%= 表达式 %>

2. 程序代码标签

<% java程序代码 %>

```
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
  <h1>Hello, JSP!</h1>
  <% int i = 5, j = 10;%>
  <table border="1px">
    <tr>
      <td><%= i %></td>
      <td><%= j %></td>
    </tr>
  </table>
  <font style="font-size:24px;color:red"><%= i %></font>
</body>
</html>
```

Java变量与表格标签相结合

Java变量与字体标签与样式相结合

什么是JSP

□ JSP的优点与缺陷

优点：

- 方便在页面中显示数据，非常容易与html标签、样式结合；

缺陷：

- 前端页面与后端处理请求杂糅在一起，不利于维护；
- 数据与功能的封装不方便。

1

什么是JSP

2

MVC设计模式

3

一个综合案例

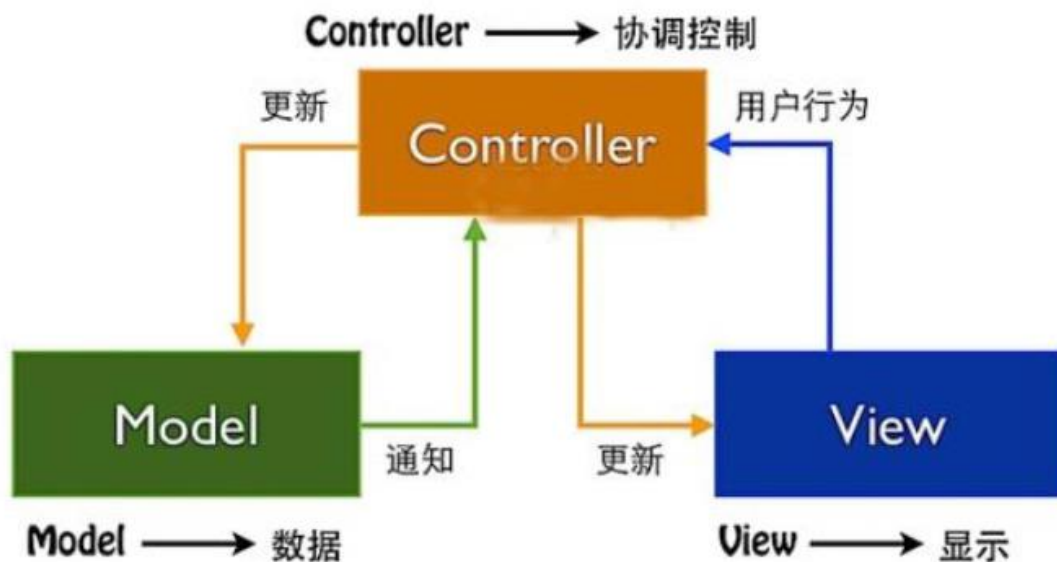
MVC设计模式

□ 什么是MVC

M: Model（模型层），将数据抽象成对象模型，负责数据的封装与处理；

V: View（视图层），一般指用户界面，负责接收用户输入的数据并将结果进行展示；

C: Controller（控制层），获取请求中的数据，向模型层发送数据与处理任务，接收数据处理结果。



Model: 数据实体类、
数据操作工具类
JavaBean

View: JSP页面

Controller: Servlet

JavaBean + JSP + Servlet

1

什么是JSP

2

MVC设计模式

3

一个综合案例

一个综合案例

□ 案例任务描述

有一个登录页面，用于学生登录（学号和密码需要查询数据库）。如果登录失败，需要回到登录页面（给出提示信息）。如果登录成功，页面进行跳转，在成功页面上显示该学生的其它相关信息（用表格进行显示）

学生在数据库中的信息（学号、姓名、密码、出生年月、
年级、籍贯、政治面貌、专业）

一个综合案例

□ 预备知识1：Request域对象

– 什么是域对象？

域对象是指一个对象可以存储一定范围内的所有数据，通过该对象就能获取和存储数据。域指的是一定的范围。

– 什么是Request域对象？

Request域对象其实就是从客户端浏览器向服务器发送的一次请求信息的封装。那么实质上向Request中所保存的数据有效期也是一次请求范围。

一次请求范围：从客户端浏览器向服务器发送一次请求，服务器针对这次请求对浏览器作出响应。当服务器作出响应之后，请求对象就销毁了，保存在其中的数据就无效了。

一个综合案例

□ 预备知识1: Request域对象

– Request域对象的常用方法

存储数据、读取数据、移除数据

<code>Object</code>	<code>getAttribute(String name)</code>
---------------------	--

	Returns the servlet container attribute with the given name, or null if there is no attribute by that name.
--	---

<code>void</code>	<code>removeAttribute(String name)</code>
-------------------	---

	Removes the attribute with the given name from the servlet context.
--	---

<code>void</code>	<code>setAttribute(String name, Object object)</code>
-------------------	---

	Binds an object to a given attribute name in this servlet context.
--	--

一个综合案例

□ 预备知识2：请求转发与重定向

— 语法形式

1. 请求转发

通过 `ServletRequest` 对象获得 `RequestDispatcher` 对象，再利用 `RequestDispatcher` 对象的 `forward` 方法，进行请求跳转！

<code>RequestDispatcher</code>	<code>getRequestDispatcher(String path)</code> Returns a <code>RequestDispatcher</code> object that acts as a wrapper for the resource located at the given path.
<code>void</code>	<code>forward(ServletRequest request, ServletResponse response)</code> Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

2. 重定向

通过 `HttpServletResponse` 对象中的以下方法实现重定向

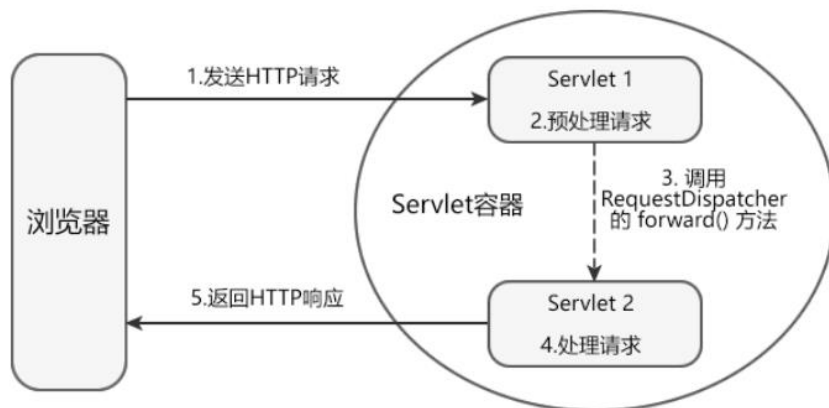
<code>void</code>	<code>sendRedirect(String location)</code> Sends a temporary redirect response to the client using the specified redirect location URL.
-------------------	--

一个综合案例

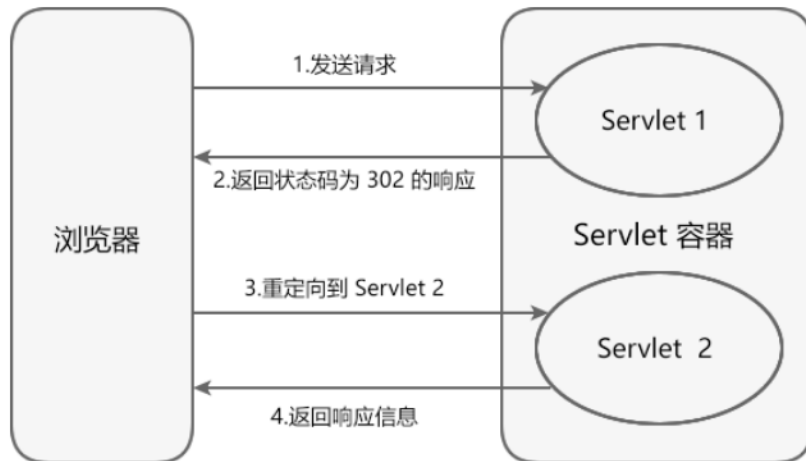
□ 预备知识2：请求转发与重定向

— 原理图解

请求转发



重定向



一个综合案例

□ 预备知识2：请求转发与重定向

— 二者区别

- ✓请求转发是一次请求一次响应，而重定向是两次请求两次响应。
- ✓请求转发地址栏不会变化的，重定向地址栏发生变化。
- ✓请求转发路径不带工程名，重定向需要带工程名路径。
- ✓请求转发只能在本网站内部，重定向可以定向到任何网站。

注：利用Request域对象保存数据时，只能使用请求转发、而不能使用重定向！！！！

如果希望跳转前后地址栏地址不会发生变化，只能使用转发；如果希望跳转前后地址栏地址会发生变化，只能使用重定向。

一个综合案例

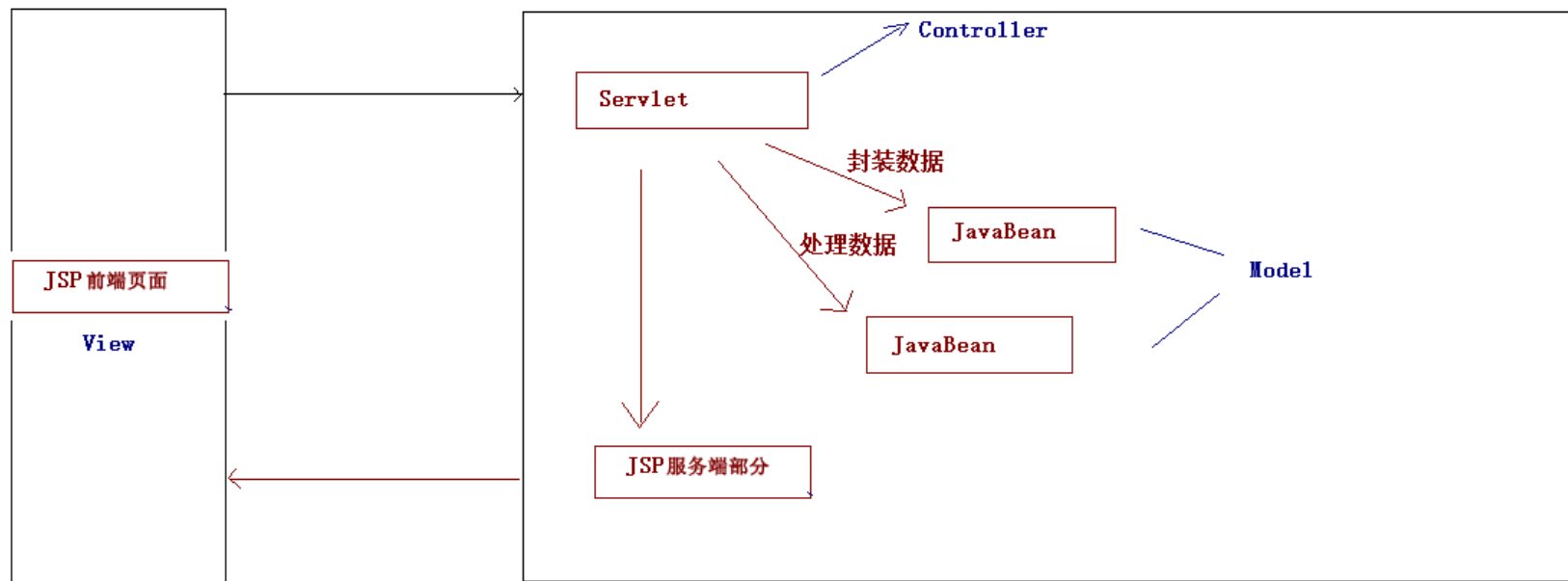
□ 具体步骤：

- 创建数据库和表，存入部分数据；
- 创建登录页面（JSP页面，View层）；
- 创建Servlet，接收请求的数据（Controller层）；
- 对数据进行抽象，创建实体类与处理的工具类（Model层）。
- 在Servlet中，对请求与响应进行控制与调度（Controller层）；
- 回到页面，将数据取出后，进行显示（JSP页面，View层）。

一个综合案例

□ 案例MVC示意图：

MVC在案例中的应用



谢谢!

Thanks for your attention!